App.js

```
import React from 'react';
import { FormProvider, RegistrationForm } from './SecureForm';

export default function App() {
  return (
    <FormProvider>
      <RegistrationForm />
    </FormProvider>
  );
}
```

SecureForm.jsx

```
import React, { useState, useEffect, useCallback, useRef, useMemo, useContext, createContext } from 'react';
import './SecureForm.css'; // Import our CSS

// 1. useContext: Global Alert/Notification System
const AlertContext = createContext();

export const FormProvider = ({ children }) => {
  const [alert, setAlert] = useState(null);

  // Helper to show an alert and automatically hide it after 3 seconds
  const showAlert = (message, type) => {
    setAlert({ message, type });
    setTimeout(() => setAlert(null), 3000);
  };
```

```jsx
  return (
    <AlertContext.Provider value={{ alert, showAlert }}>
      <div className="app-wrapper">
        {alert && (
          <div className={`alert-banner ${alert.type === 'success' ? 'alert-success' : 'alert-error'}`}>
            {alert.message}
          </div>
        )}
        {children}
      </div>
    </AlertContext.Provider>
  );
};

export const RegistrationForm = () => {
  // 2. useContext: Consume the global alert system
  const { showAlert } = useContext(AlertContext);

  // 3. useState: Manage the form's data and loading state
  const [formData, setFormData] = useState({ username: '', email: '', password: '' });
  const [isSubmitting, setIsSubmitting] = useState(false);

  // 4. useRef: Grab a reference to the first input for auto-focus
  const usernameRef = useRef(null);

  // 5. useEffect: Side effect to auto-focus the username field exactly once on load
  useEffect(() => {
    if (usernameRef.current) {
      usernameRef.current.focus();
    }
  }, []);
```

```
// 6. useCallback: Memorize the function that updates state when the user types
const handleChange = useCallback((e) => {
  const { name, value } = e.target;
  setFormData(prevData => ({ ...prevData, [name]: value }));
}, []);

// 7. useMemo: Expensive calculation simulating a password strength checker
// It only recalculates when formData.password changes, not when email or username changes.
const passwordStrength = useMemo(() => {
  const pw = formData.password;
  let score = 0;
  if (pw.length > 0) score += 1; // Typed something
  if (pw.length >= 8) score += 1; // Good length
  if (/[A-Z]/.test(pw) && /[0-9]/.test(pw)) score += 1; // Has uppercase and number
  if (/[^A-Za-z0-9]/.test(pw)) score += 1; // Has special character
  return score;
}, [formData.password]);

// Handle Form Submission
const handleSubmit = async (e) => {
  e.preventDefault();
  setIsSubmitting(true);

  // Simulate an API call to a backend database
  setTimeout(() => {
    if (passwordStrength < 3) {
      showAlert("Registration Failed: Password is too weak.", "error");
    } else {
      showAlert(`Welcome ${formData.username}! Account created securely.`, "success");
      // Clear the form
      setFormData({ username: '', email: '', password: '' });
    }
```

```
    setIsSubmitting(false);
  }, 1500);
};

return (
  <div className="form-card">
    <h2 className="form-title">System Access Node</h2>

    <form onSubmit={handleSubmit}>
      <div className="form-group">
        <label className="form-label">Username</label>
        <input
          ref={usernameRef} // Connect the useRef here
          type="text"
          name="username"
          value={formData.username}
          onChange={handleChange}
          className="form-input"
          required
        />
      </div>

      <div className="form-group">
        <label className="form-label">Secure Email</label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={handleChange}
          className="form-input"
          required
        />
```

```jsx
      </div>

      <div className="form-group">
        <label className="form-label">Password</label>
        <input
          type="password"
          name="password"
          value={formData.password}
          onChange={handleChange}
          className="form-input"
          required
        />

        {/* Password Strength Visualizer */}
        <div className="strength-meter">
          {[1, 2, 3, 4].map(level => (
            <div
              key={level}
              className={`strength-bar ${passwordStrength >= level ? `bar-filled-${passwordStrength}` : ''}`}
            />
          ))}
        </div>
      </div>

      <button
        type="submit"
        className="btn-submit"
        disabled={isSubmitting || !formData.username || !formData.email || !formData.password}
      >
        {isSubmitting ? 'Encrypting & Sending...' : 'Initialize Registration'}
      </button>
    </form>
```

```
    </div>
  );
};
```

SecureForm.css

```
.app-wrapper {
  min-height: 100vh;
  background-color: #0f172a;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 1rem;
  font-family: system-ui, -apple-system, sans-serif;
  color: #e2e8f0;
}

/* Global Alert System */
.alert-banner {
  position: fixed;
  top: 1rem;
  padding: 1rem 2rem;
  border-radius: 0.5rem;
  font-weight: bold;
  animation: slideDown 0.3s ease-out;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}
.alert-success { background-color: #065f46; color: #d1fae5; border: 1px solid #10b981; }
.alert-error { background-color: #7f1d1d; color: #fee2e2; border: 1px solid #ef4444; }

@keyframes slideDown {
```

```
  from { transform: translateY(-100%); opacity: 0; }
  to { transform: translateY(0); opacity: 1; }
}

/* Form Container */
.form-card {
  background-color: #1e293b;
  padding: 2rem;
  border-radius: 0.75rem;
  border: 1px solid #334155;
  box-shadow: 0 20px 25px -5px rgba(0, 0, 0, 0.5);
  width: 100%;
  max-width: 400px;
}
.form-title {
  font-size: 1.5rem;
  margin-top: 0;
  margin-bottom: 1.5rem;
  color: #38bdf8;
  border-bottom: 1px solid #334155;
  padding-bottom: 0.5rem;
}

/* Inputs */
.form-group {
  margin-bottom: 1.25rem;
  display: flex;
  flex-direction: column;
}
.form-label {
  font-size: 0.875rem;
  margin-bottom: 0.5rem;
```

```
  color: #94a3b8;
}
.form-input {
  padding: 0.75rem;
  background-color: #0f172a;
  border: 1px solid #475569;
  border-radius: 0.375rem;
  color: white;
  font-size: 1rem;
  transition: border-color 0.2s;
}
.form-input:focus {
  outline: none;
  border-color: #38bdf8;
}

/* Password Strength Meter */
.strength-meter {
  display: flex;
  gap: 0.25rem;
  margin-top: 0.5rem;
}
.strength-bar {
  height: 0.25rem;
  flex: 1;
  background-color: #334155;
  border-radius: 0.125rem;
  transition: background-color 0.3s;
}
.bar-filled-1 { background-color: #ef4444; } /* Red */
.bar-filled-2 { background-color: #eab308; } /* Yellow */
.bar-filled-3 { background-color: #22c55e; } /* Green */
```

```css
.bar-filled-4 { background-color: #10b981; } /* Emerald */

/* Submit Button */
.btn-submit {
  width: 100%;
  padding: 0.75rem;
  margin-top: 1rem;
  background-color: #0284c7;
  color: white;
  border: none;
  border-radius: 0.375rem;
  font-size: 1rem;
  font-weight: bold;
  cursor: pointer;
  transition: background-color 0.2s;
}
.btn-submit:hover:not(:disabled) { background-color: #0369a1; }
.btn-submit:disabled {
  background-color: #475569;
  cursor: not-allowed;
  opacity: 0.7;
}
```